

Locks

2025 Semester 1 SOFTENG 370: Operating Systems
Talia Xu

Lecture 20
2.0.0

Our Next Complication

Let's create a program that spawns 8 threads

Each thread increments the same variable 10,000 times

What should the final value of the variable be? 80,000 .

The initial value of the variable is 0

Thread Racing



```
#define NUM_THREADS 8
static int counter = 0;

void* run(__attribute__((unused)) void* arg) {
    for (int i = 0; i < 10000; ++i)
        ++counter;
    return NULL;
}

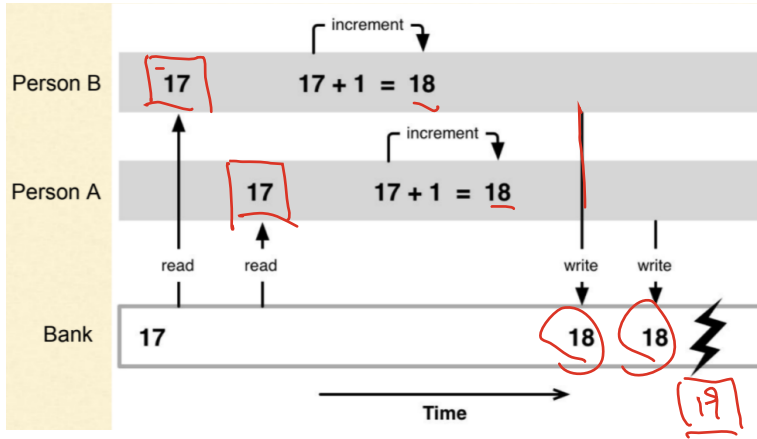
int main(void) {
    pthread_t thread[NUM_THREADS];
    for (int i = 0; i < NUM_THREADS; ++i)
        pthread_create(&thread[i], NULL, &run, NULL);
    for (i = 0; i < NUM_THREADS; ++i)
        pthread_join(thread[i], NULL);
    printf("counter = %i\n", counter);
    return 0;
}
```

Aside / Review: What about processes?

```
#define NUM_PROCESSES 8  
static int counter = 0;
```

```
int main(void) {  
    8 -  
    for (int i = 0; i < NUM_PROCESSES; ++i) {  
        start-routine.  
        pid_t pid = fork();  
        if (pid == 0) {  
            for (int j = 0; j < 10000; ++j) {  
                ++counter;  
            }  
            exit(0);  
        }  
    }  
    for (int i = 0; i < NUM_PROCESSES; ++i)  
        wait(NULL);  
  
    printf("counter = %d\n", counter);  
    return 0;  
}
```

Bank Example



Data Races Can Occur When Sharing Data

A data race is when two concurrent actions access the same variable and at least one of them is a write operation

Atomic Operations are Indivisible

Any atomic instruction you may assume happens all at once

This means you can not preempt it

However, between two atomic instructions, you may be preempted

Three Address Code (TAC) is Intermediate Code Used by Compilers

TAC is mostly used for analysis and optimization by compilers

Statements represent one fundamental operation (assume each is atomic)

Useful to reason about data races and easier to read than assembly

Statements have the form: $result := operand_1 \ operator \ operand_2$

GIMPLE is the TAC used by gcc

To see the GIMPLE representation of your compilation use:

`-fdump-tree-gimple` flag

To see all the three address code generated by the compiler (gcc) use:

`-fdump-tree-all` flag

GIMPLE is easier to reason about your code at a low-level without assembly

Data Race

Instead of count, we'll look at pcount (the pointer to count, which is a global)

The GIMPLE is the following:

→ D.1 = *pcount; counter;
→ D.2 = D.1 + 1;
→ *pcount = D.2; ← 1
 3 executions

Assuming that two threads execute this once each and initially *pcount = 0
What are the possible values of *pcount?

Data Races

Let's call the read and write from thread 1 R1 and W1 (R2 and W2 from thread 2)

We'll assume no re-ordering of instructions: always read then write in a thread

All possible orderings:

$T_1 : T_2$
 $D.1 = *pcount;$ R
 $D.2 = D.1 + 1;$ ← stack
 $*pcount = D.2;$ W
 O

Order				*pcount
R1	W1	R2	W2	2.
R1	R2	W1	W2	1
R1	R2	W2	W1	1
R2	W2	R1	W1	2.
R2	R1	W2	W1	1
R2	R1	W1	W2	1

1/3 correct
 2/3 incorrect.

Locks: The Basic Idea

```
lock_t mutex; // A lock is just a variable
```

```
...
```

```
...
```

```
 lock(&mutex); // Acquire the lock
```

```
count = count + 1
```

```
unlock(&mutex); // Release the lock
```

R - atomic
↑
preemption
↓
W - atomic

atomic

{ R
W

Only one thread can hold a lock at a time.

This thread is the owner of the lock.

Other threads wait until the lock is free again.

Pthread Locks

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  
pthread_mutex_lock(&lock);  
// protected code  
pthread_mutex_unlock(&lock);
```

Everything within the lock and unlock is protected

There's also a pthread_mutex_trylock if needed

Example of pthread lock

8 threads

```
#define NUM_ITERATIONS 100000
```

```
int counter = 0;
```

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

```
void *worker(void *arg) {
```

```
    for (int i = 0; i < NUM_ITERATIONS; ++i) {
```

```
        pthread_mutex_lock(&lock);
```

```
        counter++;
```

```
        pthread_mutex_unlock(&lock);
```

```
    }
```

```
}
```

```
int main(void){
```

```
    pthread_t t1, t2;
```

```
    pthread_create(&t1, NULL, worker, NULL);
```

```
    pthread_create(&t2, NULL, worker, NULL);
```

```
    pthread_join(t1, NULL); pthread_join(t2, NULL);
```

```
    printf("Final counter value: %d\n", counter);
```

```
    return 0;
```

```
}
```

i=0 thread 1 counter++;

i=0 t2. counter++;

i=1 t1.

What could happen with locks

```
pthread_mutex_t a = PTHREAD_MUTEX_INITIALIZER;  
pthread_mutex_t b = PTHREAD_MUTEX_INITIALIZER;
```

```
int main() {  
    pthread_t x, y;  
    pthread_create(&x, NULL, t1, NULL);  
    pthread_create(&y, NULL, t2, NULL);  
    pthread_join(x, NULL);  
    pthread_join(y, NULL);  
    return 0;  
}
```

```
void* t1(void* _) {  
    pthread_mutex_lock(&a);  
    pthread_mutex_lock(&b);  
    pthread_mutex_unlock(&b);  
    pthread_mutex_unlock(&a);  
    return NULL;  
}
```

```
void* t2(void* _) {  
    pthread_mutex_lock(&b);  
    pthread_mutex_lock(&a);  
    pthread_mutex_unlock(&a);  
    pthread_mutex_unlock(&b);  
    return NULL;  
}
```

A Critical Section Means Only One Thread Executes Instructions

Safety (aka mutual exclusion)

There should only be a single thread in a critical section at once

Liveness (aka progress)

If multiple threads reach a critical section, one must proceed

The critical section can't depend on outside threads

You can mess up and deadlock (threads don't make progress)

Bounded waiting (aka starvation-free)

A waiting thread must eventually proceed

Critical Sections Should Also Have Minimal Overhead

Efficient

You don't want to consume resources while waiting

Fair

You want each thread to wait approximately the same time

Simple

It should be easy to use, and hard to misuse

How do we build a lock?

How can we build an efficient lock?

You Could Use a Lock to Implement Critical Sections

Assuming a uniprocessor operating system, your implementation could be:

```
void lock() {  
    disable_interrupts();  
}  
void unlock() {  
    enable_interrupts();  
}
```

This would disable concurrency (assuming it ignores signals and interrupts)

This does not work on multiprocessors

Let's Try to Implement a Lock in Software

```
void init(int *l) {  
    *l = 0;  
}  
void lock(int *l) {  
    while (*l == 1);  
    *l = 1;  
}  
void unlock(int *l) {  
    *l = 0;  
}
```

What's the issue with this implementation?

Let's Try to Implement a Lock in Software

```
void init(int *l) {  
    *l = 0;  
}  
void lock(int *l) {  
    while (*l == 1);  
    *l = 1;  
}  
void unlock(int *l) {  
    *l = 0;  
}
```

What's the issue with this implementation?

It's not safe (both threads can be in the critical section)

It's not efficient, it wastes CPU cycles (busy wait)

You Can Implement Locks in Software with Minimal Hardware

Your hardware requirements just have to ensure:

- Loads and stores are atomic
- Instructions execute in order

There are 2 main algorithms you could use:

Peterson's algorithm and Lamport's bakery algorithm

However, they don't scale well, and processors execute out-of-order

Let's Assume a Magical Atomic Function — compare_and_swap

compare_and_swap(int *p, int old, int new) is atomic

It returns the original value pointed to

It only swaps if the original value equals old, and changes it to new

Let's give it another shot:

```
void init(int *l) {  
    *l = 0;  
}  
void lock(int *l) {  
    while (compare_and_swap(l, 0, 1));  
}  
void unlock(int *l) {  
    *l = 0;  
}
```

What We Implement is Essentially a Spinlock

Compare and swap is a common atomic hardware instruction

On x86 this is the `cmpxchg` instruction (compare and exchange)

However, it still has this “busy wait” problem

Consider a uniprocessor system, if you can't get the lock, you should yield
Let the kernel schedule another process, that may free the lock

On a multiprocessor machine, you could try again

Let's Add a Yield

```
void lock(int *l) {  
    while (compare_and_swap(l, 0, 1)) {  
        thread_yield();  
    }  
}
```

Now we have a **thundering herd** problem

Multiple threads may be waiting on the same lock

We have no control over who gets the lock next

We need to be able to reason about it (FIFO is okay)

We Can Add a Wait Queue to the Lock

```
void lock(int *l) {  
    while (compare_and_swap(l, 0, 1)) {  
        // add myself to the lock wait queue  
        thread_sleep();  
    }  
}  
void unlock(int *l) {  
    *l = 0;  
    if (/* threads in wait queue */) {  
        // wake up one thread  
    }  
}
```

There are 2 issues with this:

1. lost wakeup, and
2. the wrong thread gets the lock

Lost Wakeup Example

```
1 void lock(int *l) {  
2     while (compare_and_swap(l, 0, 1)) {  
3         // add myself to the wait queue  
4         thread_sleep();  
5     }  
6 }  
7 void unlock(int *l) {  
8     *l = 0;  
9     if (/* threads in wait queue */) {  
10        // wake up one thread  
11    }  
12 }
```

Assume we have thread 1 (T1) and thread 2 (T2), thread 2 holds the lock
T1 runs line 2 and fails, swap to T2 that runs lines 10-12, T1 runs lines 3 -4
T1 will never get woken up!

Wrong Thread Getting the Lock Example

```
1 void lock(int *l) {  
2     while (compare_and_swap(l, 0, 1)) {  
3         // add myself to the wait queue  
4         thread_sleep();  
5     }  
6 }  
7 void unlock(int *l) {  
8     *l = 0;  
9     if (/* threads in wait queue */) {  
10        // wake up one thread  
11    }  
12 }
```

Assume we have T1, T2, and T3. T2 holds the lock, T3 is in queue.
T2 runs line 9, swap to T1 which runs line 2 and succeeds
T1 just stole the lock from T3!

We Can Use Two Variables to Fix This (One to Guard)

```
typedef struct { int lock; int guard;
                queue_t *q; } mutex_t;
```

```
void lock(mutex_t *m) {
    while (
        compare_and_swap(m->guard, 0, 1)
    );
    if (m->lock == 0) {
        m->lock = 1; // acquire mutex
        m->guard = 0;
    } else {
        enqueue(m->q, self);
        m->guard = 0;
        thread_sleep();
        // wakeup transfers the lock here
    }
}
```

```
void unlock(mutex_t *m) {
    while (
        compare_and_swap(m->guard, 0, 1)
    );
    if (queue_empty(m->q)) {
        // release lock, no one needs it
        m->lock = 0;
    }
    else {
        // direct transfer mutex
        // to next thread
        thread_wakeup(dequeue(m->q));
    }
    m->guard = 0;
}
```

There's **STILL** A Data Race

After a thread calls `lock`, it could get interrupted right before the `thread_sleep`

However, it's been added to the wait queue, so `thread_wakeup` would try to wake up a thread that's not sleeping yet (we know it's about to)

We could simply retry the call to `thread_wakeup` until the thread finally calls `thread_sleep`

Remember What Causes a Data Race

A data race is when two concurrent actions access the same variable and at least one of them is a **write**

We could have any many readers as we want

We don't need a mutex as long as nothing writes at the same time

We need different lock modes for reading and writing

Read-Write Locks

With mutexes/spinlocks, you have to lock the data,
even for a read since you don't know if a write could happen

Reads can happen in parallel, as long as there's no write

Multiple threads can hold a read lock (`pthread_rwlock_rdlock`),
but only one thread may hold a write lock (`pthread_rwlock_wrlock`)
and will wait until the current readers are done

We Can Use A Guard To Keep Track of Readers

```
typedef struct {
    int nreader;
    lock_t guard;
    lock_t lock;
} rwlock_t;

void write_lock(rwlock_t *l) (
    lock(&l->lock);
}

void write_unlock(rwlock_t *l) (
    unlock(&l->lock);
}
```

```
void read_lock(rwlock_t *l) (
    lock(&l->guard);
    ++nreader;
    if (nreader == 1) { // first reader
        lock(&l->lock);
    }
    unlock(&l->guard);
}

void read_unlock(rwlock_t *l) (
    lock(&l->guard);
    --nreader;
    if (nreader == 0) { // last reader
        unlock(&l->lock);
    }
    unlock(&l->guard);
}
```

We Want Critical Sections to Protect Against Data Races

We should know what data races are, and how to prevent them:

- Mutex or spinlocks are the most straightforward locks
- We need hardware support to implement locks
- We need some kernel support for wake up notifications
- If we know we have a lot of readers, we should use a read-write lock